AIM:Naive Bayes' Classifier
Implement the Naive Bayes' algorithm for classification.
Train a Naive Bayes' model using a given dataset and calculate class probabilities.
Evaluate the accuracy of the model on test data and analyze the results.

CODE:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

# 1. Load and Clean Dataset
df = pd.read_csv("C:/Users/hrane/Downloads/online food delivery_107.csv")
df.columns = df.columns.str.strip()
df = df.apply(lambda x: x.str.strip() if x.dtype == "object" else x)

# 2. Preprocessing
df["Output"] = df["Output"].map({"Yes": 1, "No": 0})

# Drop target and leaky column
X = df.drop(["Output", "Feedback"], axis=1, errors="ignore")
y = df["Output"]

# Use One-Hot Encoding (Fixes the out-of-bounds index mismatch error)
X = pd.get_dummies(X, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Train Naive Bayes Model
model = MultinomialNB()
model.fit(X_train, y_train)

# 4. Extract Calculated Class Prior Probabilities
class_prior_probs = np.exp(model.class_log_prior_)

print("=" * 50)
print("--- CALCULATED CLASS PRIOR PROBABILITIES ---")
print(f"Probability of 'No'  (Class 0): {class_prior_probs[0]:.4f}")
print(f"Probability of 'Yes' (Class 1): {class_prior_probs[1]:.4f}")
print("=" * 50)

# 5. Evaluate Model Accuracy
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred) * 100
```


RIYA T107

```
print(f"\nModel Test Accuracy: {accuracy:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=["No", "Yes"]))

# 6. Generate Class Probability Distribution Graph
plt.figure(figsize=(6, 4))
classes = ["No (Output=0)", "Yes (Output=1)"]
colors = ["#ff9999", "#66b3ff"]

bars = plt.bar(classes, class_prior_probs, color=colors, width=0.4)

plt.title("Naive Bayes: Prior Class Probability Distribution")
plt.ylabel("Probability Balance")
plt.ylim(0, 1.1)
plt.grid(axis="y", linestyle="--", alpha=0.5)

# Add probability value text tags over the chart bars
for bar in bars:
    yval = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        yval + 0.02,
        f"{yval:.4f}",
        ha="center",
        va="bottom",
        fontweight="bold",
    )

plt.tight_layout()
plt.show()
```

OUTPUT:

RIYA T107

```
========== RESTART: C:\Users\hrane\Documents\AI_R.
==================================================
--- CALCULATED CLASS PRIOR PROBABILITIES ---
Probability of 'No'  (Class 0): 0.2258
Probability of 'Yes' (Class 1): 0.7742
==================================================

Model Test Accuracy: 91.03%

Classification Report:
              precision    recall  f1-score   support

          No       1.00      0.59      0.74        17
         Yes       0.90      1.00      0.95        61

    accuracy                           0.91        78
   macro avg       0.95      0.79      0.84        78
weighted avg       0.92      0.91      0.90        78
```



RIYA T107